

U2-AT2 SOLUTION

1. SOLUTION:

The AI-powered drone routing problem can be solved using Greedy Best-First Search (GBFS) and A* Search algorithms. GBFS selects the next path based only on the heuristic value $h(n)$ (straight-line distance), making it very fast and suitable for emergency situations, but it ignores the actual travel cost and may choose unsafe or blocked routes. In contrast, A* Search uses the evaluation function $f(n)=g(n)+h(n)$, where $g(n)$ is the actual travel cost and $h(n)$ is the estimated distance to the destination, enabling it to find the safest and most cost-effective path. The accuracy of the heuristic directly affects both algorithms; an accurate heuristic improves search speed, while a poor heuristic can lead GBFS to inefficient routes and cause A* to explore more nodes, although A* still guarantees an optimal solution if the heuristic is admissible. In dynamic environments, sudden road blockages force GBFS to restart its search, whereas A* can update costs and efficiently replan an alternative route. Therefore, A* Search is the most suitable algorithm because it balances real-time performance, optimality, adaptability, and safety, making it ideal for emergency medicine delivery in flood-affected regions.

2. SOLUTION:

The smart city traffic signal optimization problem is formulated as an optimization problem where the objective is to minimize total vehicle waiting time, fuel consumption, and traffic congestion while continuously adapting to changing traffic conditions. Traditional search algorithms are inefficient because the search space is extremely large and dynamic, making exhaustive exploration impractical. Hill Climbing improves the current traffic signal configuration by repeatedly selecting neighboring solutions with better performance, but it can become trapped in local maxima, plateaus, and ridges, preventing it from finding the global optimum. Simulated Annealing overcomes this limitation by occasionally accepting worse solutions to escape local optima, while the Genetic Algorithm (GA) uses selection, crossover, and mutation to evolve multiple candidate solutions simultaneously, allowing it to explore a much larger search space efficiently. Since GA provides better scalability, adaptability, and optimization performance in complex and dynamic environments, it is the most suitable approach for managing traffic signals across hundreds of intersections in a smart city.

3. SOLUTION:

The Mars rover exploration problem requires an Online Search Agent because the rover operates in an unknown environment without a complete map and must make decisions while exploring. Unlike offline search, which assumes full knowledge of the environment before execution, online search continuously senses nearby terrain,

updates its internal map, and chooses the safest next action based on newly acquired information. The rover faces challenges such as partial observability, where hazards like rocks and craters cannot be detected until they are within sensor range, and dynamic environmental changes, including dust storms and unstable terrain that may alter safe paths. As the rover explores, it builds an internal model by marking safe routes, hazards, and explored locations, enabling better future decisions. Compared with uninformed and informed search algorithms, online search is more suitable because it adapts to uncertainty, learns during execution, and supports real-time autonomous navigation. Therefore, an Online Search Agent is the best choice due to its adaptability, safety, and ability to operate effectively under uncertain conditions.

4. SOLUTION:

The university exam timetable generation problem is modeled as a Constraint Satisfaction Problem (CSP), where the exam subjects are treated as variables, available time slots represent domains, and constraints ensure that no student has overlapping exams, exam hall capacities are respected, subject ordering requirements are maintained, faculty availability is considered, and special accommodations are provided. Backtracking Search solves the problem by assigning one subject to a valid time slot, checking all constraints, and returning to previous assignments whenever a conflict occurs until a valid timetable is produced. To improve efficiency, Forward Checking, a constraint propagation technique, removes invalid time slots from the remaining subjects immediately after each assignment, reducing unnecessary searches and minimizing backtracking. As the number of courses, students, and constraints increases, the scheduling problem becomes highly complex, making Backtracking combined with Forward Checking the most effective strategy because it reduces the search space, detects conflicts early, and efficiently generates scalable and valid exam timetables.

5. SOLUTION:

The real-time strategy game AI problem is modeled using the Minimax algorithm, where the AI acts as the maximizing player and assumes that the human opponent always chooses the best possible minimizing move. The algorithm evaluates all possible game states to determine the optimal action, but as the branching factor and search depth increase, the computation becomes expensive due to the exponential growth of the game tree. Alpha-Beta Pruning improves the efficiency of Minimax by eliminating branches that cannot influence the final decision using Alpha (α) and Beta (β) values, thereby reducing the number of nodes evaluated while still producing the same optimal result. The AI uses an evaluation function based on factors such as resources, army strength, territory control, enemy strength, and unit losses to estimate the quality of each game state. Since Alpha-Beta Pruning significantly reduces computation time without sacrificing decision quality, Minimax with Alpha-Beta Pruning is the most suitable strategy for balancing optimality, computational efficiency, and real-time performance in complex strategy games.